In [4]:
```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()

df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['species'] = iris.target_names[iris.target]

print("DataFrame head:")
display(df.head())
print("\nDataFrame info:")
df.info()
```

DataFrame head:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

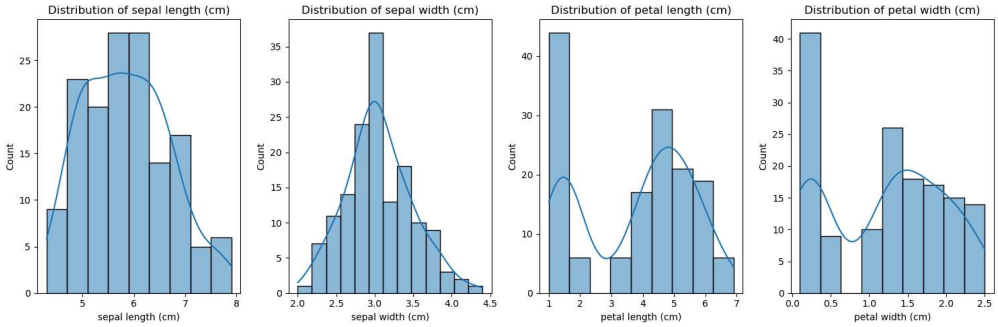
In [5]:
```
print("Descriptive Statistics:")
display(df.describe())
```

```
Descriptive Statistics:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

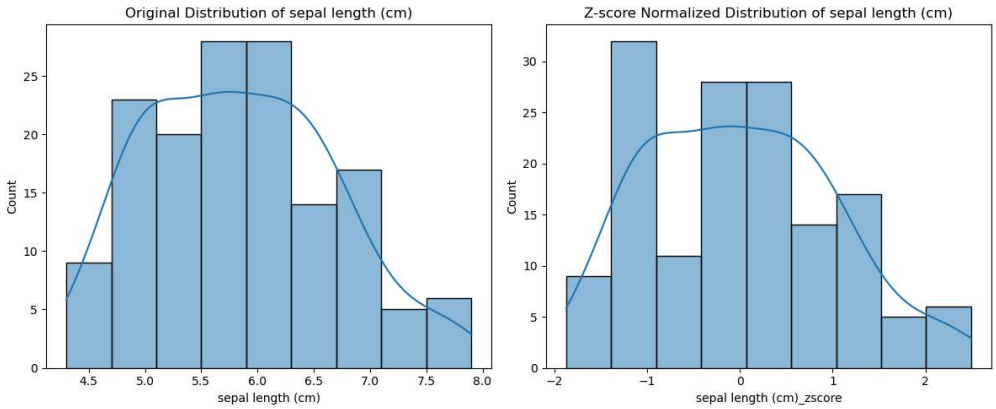
In [6]:
```
import matplotlib.pyplot as plt
import seaborn as sns

print("Skewness of each numerical feature:")
numerical_features = df.select_dtypes(include=['float64', 'int64'])
display(numerical_features.skew())

fig, axes = plt.subplots(1, len(iris.feature_names), figsize=(15, 5))
for i, feature in enumerate(iris.feature_names):
    sns.histplot(df[feature], kde=True, ax=axes[i])
    axes[i].set_title(f'Distribution of {feature}')
plt.tight_layout()
plt.show()
```

```
Skewness of each numerical feature:


sepal length (cm)    0.314911
sepal width (cm)     0.318966
petal length (cm)   -0.274884
petal width (cm)    -0.102967
dtype: float64
```



In [7]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

feature_to_normalize = 'sepal length (cm)'

df[f'{feature_to_normalize}_zscore'] =
scaler.fit_transform(df[[feature_to_normalize]])

print(f"Original '{feature_to_normalize}' descriptive statistics:")
display(df[feature_to_normalize].describe())

print(f"\nZ-score normalized '{feature_to_normalize}' descriptive
statistics:")
display(df[f'{feature_to_normalize}_zscore'].describe())

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sns.histplot(df[feature_to_normalize], kde=True, ax=axes[0])
axes[0].set_title(f'Original Distribution of {feature_to_normalize}')
sns.histplot(df[f'{feature_to_normalize}_zscore'], kde=True, ax=axes[1])
axes[1].set_title(f'Z-score Normalized Distribution of
{feature_to_normalize}')
plt.tight_layout()
plt.show()
```

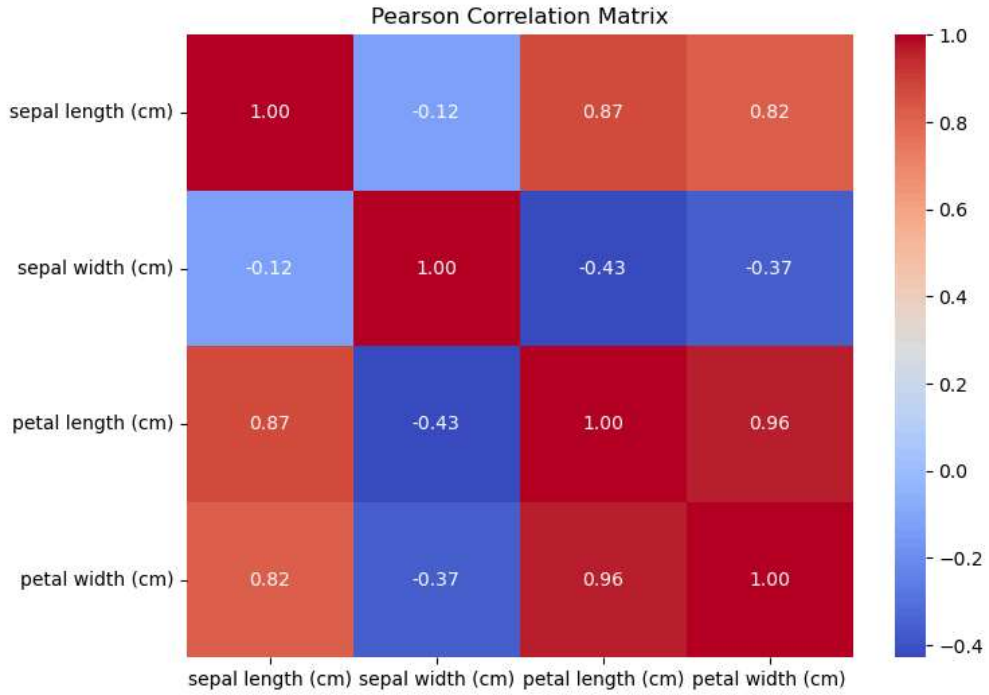
```
Original 'sepal length (cm)' descriptive statistics:

count    150.000000
mean       5.843333
std        0.828066
min        4.300000
25%        5.100000
50%        5.800000
75%        6.400000
max        7.900000
Name: sepal length (cm), dtype: float64


Z-score normalized 'sepal length (cm)' descriptive statistics:

count    1.500000e+02
mean    -4.736952e-16
std      1.003350e+00
min     -1.870024e+00
25%     -9.006812e-01
50%     -5.250608e-02
75%      6.745011e-01
max      2.492019e+00
Name: sepal length (cm)_zscore, dtype: float64
```


Original Distribution of sepal length (cm)
Count
sepal length (cm)
Z-score Normalized Distribution of sepal length (cm)
Count
sepal length (cm)_zscore

In [8]:
```
numerical_df = df[iris.feature_names]

print("Pearson Correlation Matrix:")
pearson_corr = numerical_df.corr(method='pearson')
display(pearson_corr)

plt.figure(figsize=(8, 6))
sns.heatmap(pearson_corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Pearson Correlation Matrix')
plt.show()

print("\nSpearman Correlation Matrix:")
spearman_corr = numerical_df.corr(method='spearman')
display(spearman_corr)

plt.figure(figsize=(8, 6))
sns.heatmap(spearman_corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Spearman Correlation Matrix')
plt.show()
```

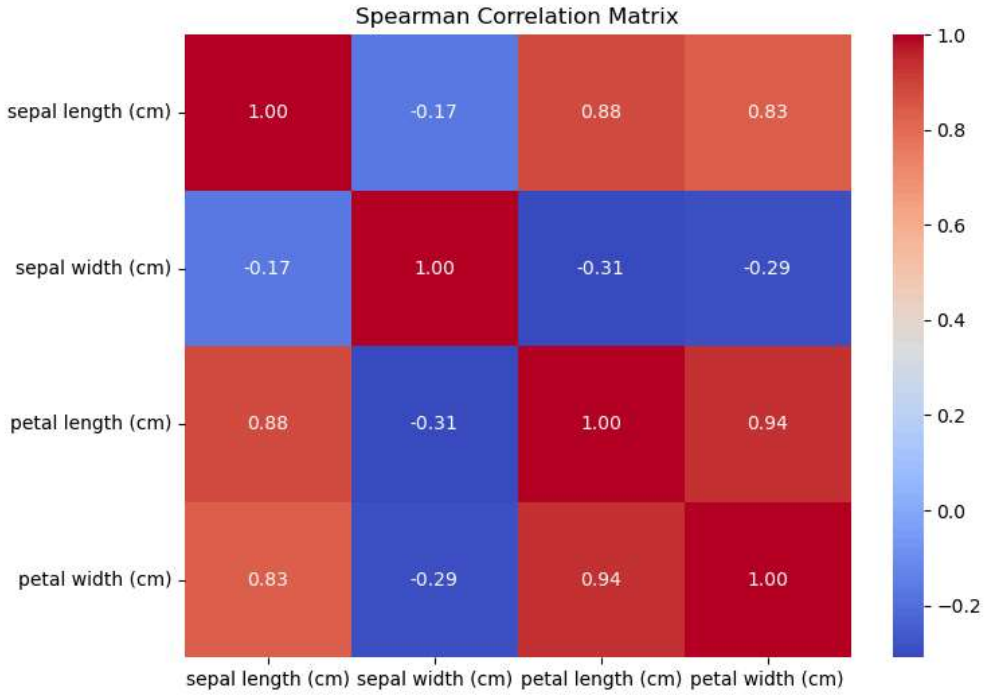
```
Pearson Correlation Matrix:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126
petal length (cm)	0.871754	-0.428440	1.000000	0.962865
petal width (cm)	0.817941	-0.366126	0.962865	1.000000



Spearman Correlation Matrix:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
sepal length (cm)	1.000000	-0.166778	0.881898	0.834289
sepal width (cm)	-0.166778	1.000000	-0.309635	-0.289032
petal length (cm)	0.881898	-0.309635	1.000000	0.937667
petal width (cm)	0.834289	-0.289032	0.937667	1.000000



In [9]:
```
df.loc[0, 'embark_town'] = 'Southampton'
df.loc[1, 'embark_town'] = 'SOUTHAMPTON'
df.loc[2, 'embark_town'] = 'Southampton'
print(df['embark_town'].unique())
```

```
['Southampton' 'SOUTHAMPTON' nan]
```

In [10]:
```
df['embark_town'] = df['embark_town'].str.title()
```

In [11]:
```
print(df['embark_town'].unique())
```

```
['Southampton' nan]
```

In [12]:
```
print(df['embark_town'].isnull().sum())
```

```
147
```

In [23]:
```
import pandas as pd
import seaborn as sns
df = sns.load_dataset('titanic')
print("Before fillna:")
print(df['embark_town'].value_counts())
df['embark_town'] = df['embark_town'].fillna('Kingstown')  # Capital K
print("\nAfter fillna:")
print(df['embark_town'].value_counts())
print(f"\nRemaining nulls: {df['embark_town'].isnull().sum()}")
```

```
Before fillna:
embark_town
Southampton    644
Cherbourg      168
Queenstown      77
Name: count, dtype: int64

After fillna:
embark_town
Southampton    644
Cherbourg      168
Queenstown      77
Kingstown        2
Name: count, dtype: int64

Remaining nulls: 0
```

In [25]:
```
print(df['embark_town'].unique())
```

```
['Southampton' 'Cherbourg' 'Queenstown' 'Kingstown']
```

In [27]:
```
duplicate_rows = df.iloc[:5]
df_dup = pd.concat([df, duplicate_rows])
print(df_dup.shape)
```

```
(896, 15)
```

In [28]:
```
# View Duplicate Rows

df_dup[df_dup.duplicated()]
```

Out[28]:

	survived	pclass	sex	age	sibsp	parch	fare	embarke
47	1	3	female	NaN	0	0	7.7500	Q
76	0	3	male	NaN	0	0	7.8958	S
77	0	3	male	NaN	0	0	8.0500	S
87	0	3	male	NaN	0	0	8.0500	S
95	0	3	male	NaN	0	0	8.0500	S
...	...	...	...	...	...	...	...	...
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S

112 rows × 15 columns

In [2]:
```
import pandas as pd
import numpy as np
data = {
'age': [22, 25, np.nan, 24, 23],
'fare': [7.25, 8.05, 7.90, 8.00, 7.50]
}
df = pd.DataFrame(data)
print(df)
```

```
   age  fare
0  22.0  7.25
1  25.0  8.05
2   NaN  7.90
3  24.0  8.00
4  23.0  7.50
```

In [3]:
```
from sklearn.impute import KNNImputer
KNN = KNNImputer (n_neighbors=3)
df_KNN = pd.DataFrame(
KNN.fit_transform(df),
columns=df.columns)
print(df_KNN)
```

```
    age  fare
0  22.0  7.25
1  25.0  8.05
2  24.0  7.90
3  24.0  8.00
4  23.0  7.50
```

In [5]:
```
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer
df = sns.load_dataset('titanic')
df['embarked_ffill'] = df['embarked'].ffill()
cols = ['age', 'fare', 'pclass', 'sibsp', 'parch']
imputer = KNNImputer(n_neighbors=5)
imputed_values = imputer.fit_transform(df[cols])
df_imputed = pd.DataFrame(imputed_values, columns=cols)
df_imputed = df_imputed.add_suffix('_knn')
df = pd.concat([df, df_imputed], axis=1)
print("Original missing values:")
print(df[['age', 'fare']].isnull().sum())
print("\nMissing values after KNN imputation:")
print(df[['age_knn', 'fare_knn']].isnull().sum())
print("\nFirst 5 rows (original vs imputed):")
print(df[['age', 'age_knn', 'fare', 'fare_knn']].head(10))
```

```
Original missing values:
age     177
fare      0
dtype: int64

Missing values after KNN imputation:
age_knn     0
fare_knn    0
dtype: int64

First 5 rows (original vs imputed):
    age  age_knn     fare  fare_knn
0  22.0     22.0   7.2500    7.2500
1  38.0     38.0  71.2833   71.2833
2  26.0     26.0   7.9250    7.9250
3  35.0     35.0  53.1000   53.1000
4  35.0     35.0   8.0500    8.0500
5   NaN     24.8   8.4583    8.4583
6  54.0     54.0  51.8625   51.8625
7   2.0      2.0  21.0750   21.0750
8  27.0     27.0  11.1333   11.1333
9  14.0     14.0  30.0708   30.0708
```

In [7]:
```
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer
df = sns.load_dataset('titanic')
df['embarked_ffill'] = df['embarked'].ffill()
cols = [
    'age',
    'fare',
    'sibsp',
    'parch',
    'pclass'
]
imputer = KNNImputer(n_neighbors=5)
df[cols] = imputer.fit_transform(df[cols])
print(df.isnull().sum())
```

```
survived           0
pclass             0
sex                0
age                0
sibsp              0
parch              0
fare               0
embarked           2
class              0
who                0
adult_male         0
deck             688
embark_town        2
alive              0
alone              0
embarked_ffill     0
dtype: int64
```

In [9]:
```
import pandas as pa
import seaborn as sns
from sklearn.impute import KNNImputer
df = sns.load_dataset('titanic')
df['embarked_ffill'] = df['embarked'].ffill()
cols = ['age', 'fare', 'sibsp', 'parch', 'pclass']
imputer = KNNImputer(n_neighbors=5)
df[cols] = imputer.fit_transform(df[cols])
print(df.isnull().sum())
```

```
survived           0
pclass             0
sex                0
age                0
sibsp              0
parch              0
fare               0
embarked           2
class              0
who                0
adult_male         0
deck             688
embark_town        2
alive              0
alone              0
embarked_ffill     0
dtype: int64
```

In [2]:
```
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
df = sns.load_dataset('titanic')
le = LabelEncoder()  # Added '=' here
df['sex_encoded'] = le.fit_transform(df['sex'])
print(df[['sex', 'sex_encoded']].head(10))
print("\nMapping: Male = 1, Female = 0")
```

```
      sex  sex_encoded
0    male            1
1  female            0
2  female            0
3  female            0
4    male            1
5    male            1
6    male            1
7    male            1
8  female            0
9  female            0

Mapping: Male = 1, Female = 0
```

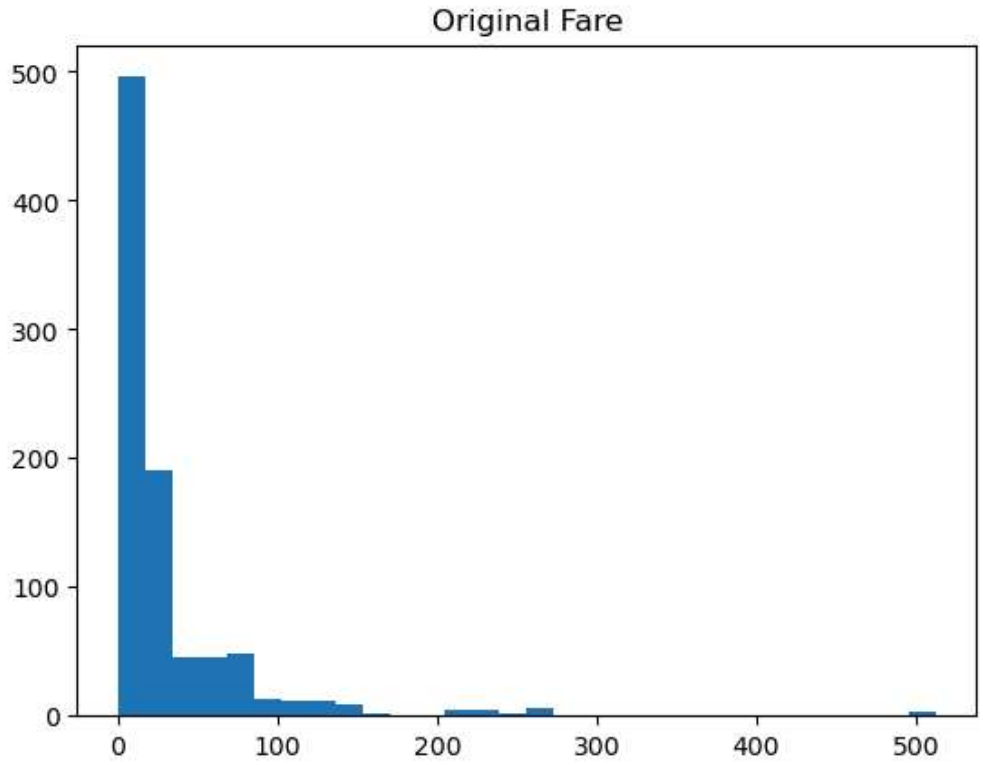
In [7]:
```
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
df = sns.load_dataset('titanic')
le = LabelEncoder()
df['sex_encoded'] = le.fit_transform(df['sex'])
df['embarked_ffill'] = df['embarked'].ffill()
df = pd.get_dummies(df, columns=['embarked'], drop_first=True)
target_map = df.groupby('pclass')['survived'].mean()
df['class_target'] = df['pclass'].map(target_map)
print("Target map (survival rate by class):")
print(target_map)
print("\n" + "="*50)
print("\nFirst 10 rows showing 'pclass' and 'class_target':")
print(df[['pclass', 'class_target']].head(10))
print("\n" + "="*50)
print(f"\nShape of dataframe: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
```

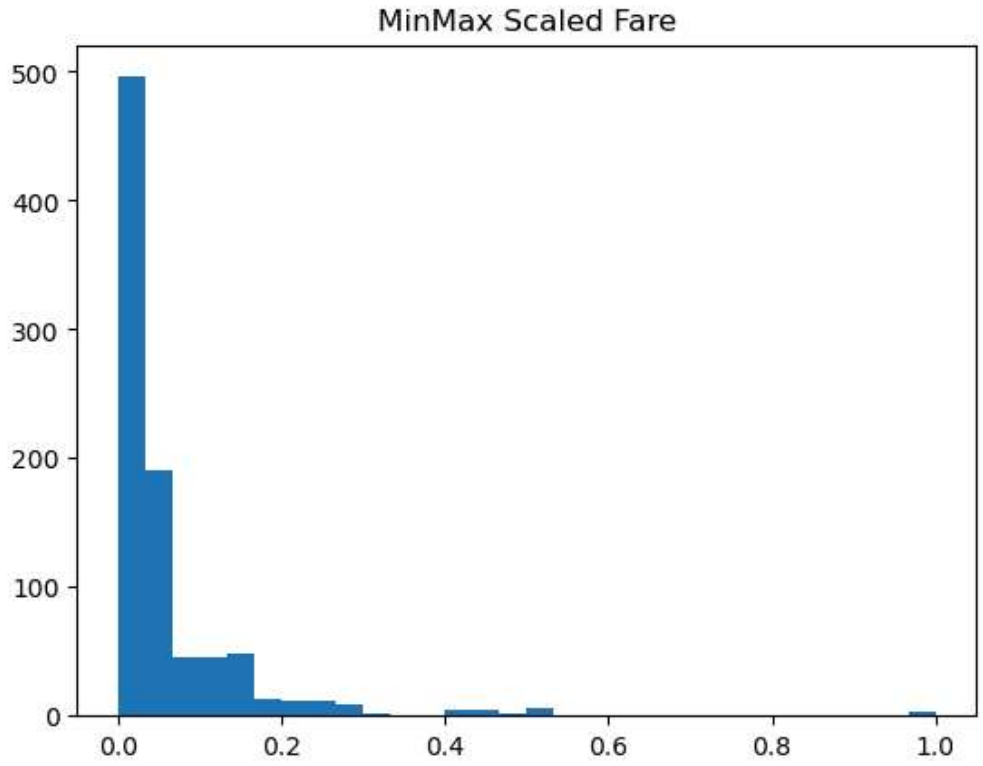
```
Target map (survival rate by class):
pclass
1    0.629630
2    0.472826
3    0.242363
Name: survived, dtype: float64

==================================================

First 10 rows showing 'pclass' and 'class_target':
   pclass  class_target
0       3      0.242363
1       1      0.629630
2       3      0.242363
3       1      0.629630
4       3      0.242363
5       3      0.242363
6       1      0.629630
7       3      0.242363
8       3      0.242363
9       2      0.472826

==================================================

Shape of dataframe: (891, 19)
Columns: ['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch',
'fare', 'class', 'who', 'adult_male', 'deck', 'embark_town', 'alive',
'alone', 'sex_encoded', 'embarked_ffill', 'embarked_Q', 'embarked_S',
'class_target']
```

In [13]:
```
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
df = sns.load_dataset('titanic')
scaler = MinMaxScaler()
df[['age_minmax', 'fare_minmax']] = scaler.fit_transform(df[['age',
'fare']])
print(df[['age_minmax', 'fare_minmax']].head())
```

```
   age_minmax  fare_minmax
0    0.271174     0.014151
1    0.472229     0.139136
2    0.321438     0.015469
3    0.434531     0.103644
4    0.434531     0.015713
```

In [15]:
```
import matplotlib.pyplot as plt
plt.hist(df['fare'].dropna(), bins=30)
plt.title("Original Fare")
plt.show()
plt.hist(df['fare_minmax'], bins=30)
plt.title("MinMax Scaled Fare")
plt.show()
```





In [ ]: